University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Ground Localization

Srikanth Nanda

20. Februar 2026

1. Feature-Based Ground Texture Localization
2. Appearance-Based Localization
3. Feature-Based Visual Odometry
4. Map-Based Absolute Localization
5. Example Code for Ground Texture Algorithms
6. Quellen

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Feature-Based Ground Texture Localization

Idea I

Ground-texture localization relies on:

- Detecting stable keypoints on asphalt, concrete, tiles Agrawal, Konolige und Blas, 2008; Bay, Tuytelaars und Van Gool, 2008; Alcantarilla, Nuevo und Bartoli, 2013.
- Extracting descriptors such as SIFT, SURF, AKAZE, BRIEF, ORB Lowe, 2004; Calonder u. a., 2010; Rublee u. a., 2011.
- Matching descriptors against a map or between frames Matas u. a., 2004.
- Pose estimation via RANSAC Desouza und Kak, 2002.

Hyperparameters I

- Number of keypoints (200–2000) Schmid, Simon und Mester, 2019.
- Descriptor size: SIFT (128), ORB (32) Lowe, 2004; Rublee u. a., 2011.
- Matching threshold: Lowe ratio 0.7–0.9 Lowe, 2004.
- RANSAC iterations (100–1000) Desouza und Kak, 2002.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Library I

- OpenCV: SIFT, SURF, ORB, BRISK, AKAZE Bradski, 2000.
- libv4l + OpenCV for downward camera capture.

Applications I

- Indoor AMRs (autonomous mobile robots) Kelly u. a., 2007.
- Industrial AGVs without infrastructure Zhang, Finkelstein und Rusinkiewicz, 2019.
- Warehouse floor navigation Chen, Vempati und Beardsley, 2018.



Abbildung: Feature-based localization pipeline

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Appearance-Based Localization

Idea I

- Uses correlation or template matching between patches Kelly u. a., 2007; Zaman, 2007.
- Computes pixel displacement using NCC/SSD Pratt, 2007.
- Converts displacement to metric motion with planar assumption.

Hyperparameters I

- Patch size (16–64 px) Aqel u. a., 2016.
- Search window radius (10–50 px).
- NCC/SSD threshold for valid match.

Library I

- OpenCV: matchTemplate(), NCC/SSD.

Applications I

- Incremental localization for factory robots Kelly u. a., 2007.
- Floor tracking for simple AGVs Zaman, 2007.

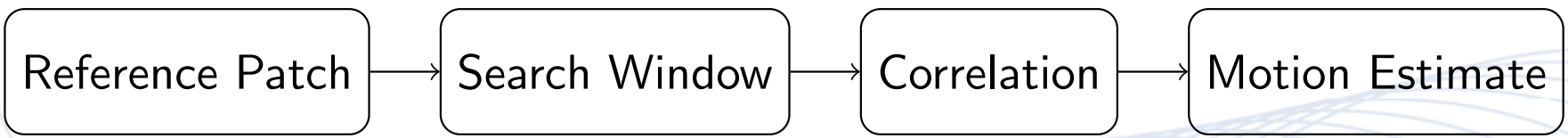


Abbildung: Appearance-based localization flow

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Feature-Based Visual Odometry

Idea I

- Detect keypoints in consecutive frames Agrawal, Konolige und Blas, 2008; Alcantarilla, Nuevo und Bartoli, 2013.
- Match descriptors (ORB, BRISK, AKAZE) Rublee u. a., 2011; Leutenegger, Chli und Siegwart, 2011.
- Estimate frame-to-frame motion with RANSAC Desouza und Kak, 2002.

Hyperparameters I

- Keypoint budget per frame (500–1500).
- Hamming distance threshold for binary descriptors.
- RANSAC inlier threshold (1–3 px).

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Library I

- OpenCV: ORB-SLAM2 style pipeline Mur-Artal und Tardós, 2017.

Applications I

- Short-range floor-based odometry Nakashima, Morio und Mu, 2019; Kozak und Alban, 2016.

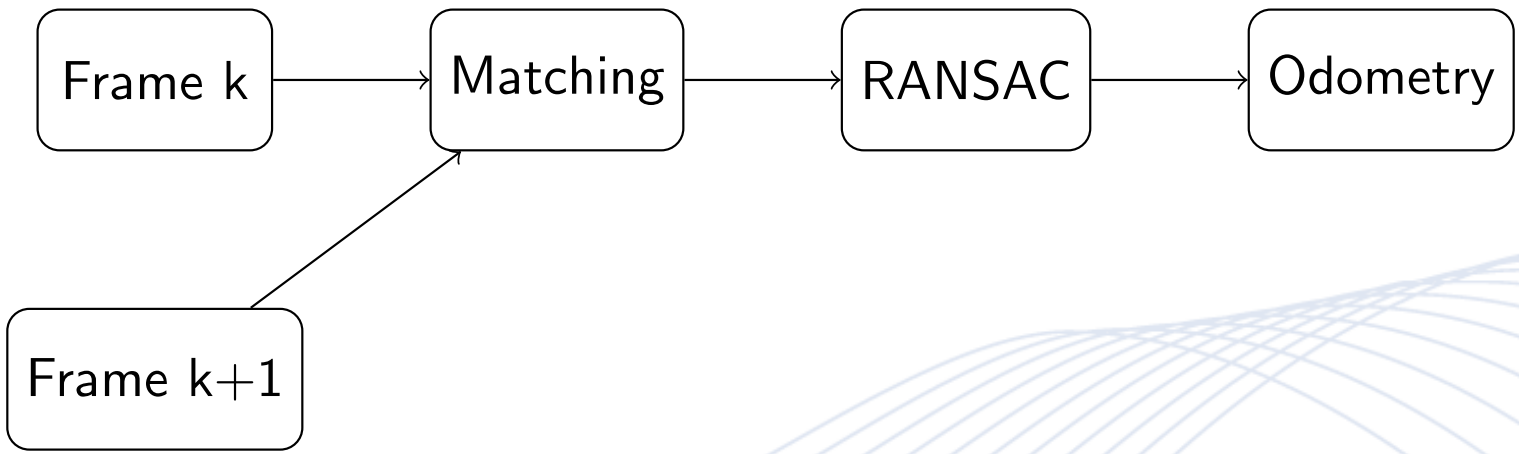


Abbildung: Feature-based visual odometry (scale=0.85)

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Map-Based Absolute Localization

Idea I

- Offline mapping of ground features Zhang, Finkelstein und Rusinkiewicz, 2019.
- Online matching of current image to map keypoints Chen, Vempati und Beardsley, 2018.
- RANSAC pose recovery Schmid, Simon und Mester, 2019.

Hyperparameters I

- Map feature density (5–20 keypoints/m^2).
- Descriptor type for map (SIFT/AKAZE) Zhang, Finkelstein und Rusinkiewicz, 2019.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Library I

- OpenCV + FLANN Bradski, 2000.

Applications I

- Autonomous warehouse robots Zhang, Finkelstein und Rusinkiewicz, 2019.
- Large-area indoor localization Cornick u. a., 2016.



Abbildung: Map-based absolute localization

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Example Code for Ground Texture Algorithms

Full File I

```
#!/usr/bin/env python3
"""
/**
 * @file Ground.py
 * @brief Ground Texture Localization Algorithms: Feature-Based, Appearance-Based,
 *        Visual Odometry, and Map-Based Localization.
 * @author Srikanth
 * @date 2025
 *
 * This module provides implementations of different ground-texture-based
 * localization algorithms using OpenCV:
 *
 *  - Feature-Based Ground Texture Localization
 *  - Appearance-Based Localization (Correlation)
 *  - Feature-Based Visual Odometry
 *  - Map-Based Absolute Localization
 *
 * Requires: OpenCV, NumPy
```

Full File II

```
 */
"""

import cv2
import numpy as np

##############################################################
# 1. FEATURE-BASED GROUND TEXTURE LOCALIZATION
##############################################################

def feature_based_localization(img1, img2, method="ORB"):
    """
    Detect & match local features between two ground texture
     images.
    Args:
        img1: reference image
        img2: query image
        method: 'SIFT', 'ORB', 'AKAZE'
    Returns:
        matches, kp1, kp2
```

Full File III

```
38      """
39      if method == "SIFT":
40          detector = cv2.SIFT_create()
41          norm = cv2.NORM_L2
42      elif method == "AKAZE":
43          detector = cv2.AKAZE_create()
44          norm = cv2.NORM_HAMMING
45      else:
46          detector = cv2.ORB_create(nfeatures=1000)
47          norm = cv2.NORM_HAMMING
48
49      kp1, des1 = detector.detectAndCompute(img1, None)
50      kp2, des2 = detector.detectAndCompute(img2, None)
51
52      matcher = cv2.BFMatcher(norm)
53      raw_matches = matcher.knnMatch(des1, des2, k=2)
54
55      # Lowe ratio test
56      good = []
57      for m, n in raw_matches:
```

Full File IV

```
58          if m.distance < 0.75 * n.distance:
59              good.append(m)
60
61      return kp1, kp2, good
62
63
64  ##################################################################
65  # 2. APPEARANCE-BASED LOCALIZATION (PATCH CORRELATION)
66  ##################################################################
67
68  def appearance_based_localization(ref, query, patch_size=32,
        search_radius=32):
69      """
70      Compute displacement between two frames using template
        matching.
71      Uses NCC (Normalized Cross Correlation).
72      """
73      h, w = ref.shape[:2]
74      cx, cy = w // 2, h // 2
75
```

Full File V

```
76      # Extract central reference patch
77      ref_patch = ref[cy - patch_size:cy + patch_size,
78                      cx - patch_size:cx + patch_size]
79
80      search = query[cy - search_radius:cy + search_radius,
81                     cx - search_radius:cx + search_radius]
82
83      res = cv2.matchTemplate(search, ref_patch, cv2.
        TM_CCOEFF_NORMED)
84      _, _, _, max_loc = cv2.minMaxLoc(res)
85
86      dx = max_loc[0] - (search_radius - patch_size)
87      dy = max_loc[1] - (search_radius - patch_size)
88
89      return dx, dy
90
91
92  ##############################################################
93  # 3. FEATURE-BASED VISUAL ODOMETRY
94  ##############################################################
```

Full File VI

```
95
96  def feature_based_vo(prev_frame, curr_frame):
97      """
98      Estimate frame-to-frame motion using feature matching +
        RANSAC.
99      """
100     orb = cv2.ORB_create(1500)
101     kp1, des1 = orb.detectAndCompute(prev_frame, None)
102     kp2, des2 = orb.detectAndCompute(curr_frame, None)
103
104     bf = cv2.BFMatcher(cv2.NORM_HAMMING)
105     raw = bf.knnMatch(des1, des2, k=2)
106
107     good = []
108     for m, n in raw:
109         if m.distance < 0.75 * n.distance:
110             good.append(m)
111
112     if len(good) < 8:
113         return None, None, None
```

Full File VII

```
114
115     pts1 = np.float32([kp1[m.queryIdx].pt for m in good])
116     pts2 = np.float32([kp2[m.trainIdx].pt for m in good])
117
118     H, mask = cv2.findHomography(pts1, pts2, cv2.RANSAC)
119
120     return H, kp1, kp2
121
122
123 ##################################################################
124 # 4. MAP-BASED ABSOLUTE LOCALIZATION
125 ##################################################################
126
127 def map_based_localization(query, map_keypoints,
        map_descriptors):
128     """
129     Query a global map of ground keypoints using FLANN.
130     """
131     detector = cv2.SIFT_create()
132     kpq, desq = detector.detectAndCompute(query, None)
```

Full File VIII

```
133
134     index_params = dict(algorithm=1, trees=5)    # KD-tree
135     search_params = dict(checks=50)
136     flann = cv2.FlannBasedMatcher(index_params,
        search_params)
137
138     raw_matches = flann.knnMatch(desq, map_descriptors, k=2)
139
140     good = []
141     for m, n in raw_matches:
142         if m.distance < 0.75 * n.distance:
143             good.append(m)
144
145     if len(good) < 8:
146         return None, kpq, []
147
148     pts_q = np.float32([kpq[m.queryIdx].pt for m in good])
149     pts_m = np.float32([map_keypoints[m.trainIdx].pt for m
        in good])
150
```

Full File IX

```
151     H, mask = cv2.findHomography(pts_q, pts_m, cv2.RANSAC)
152
153     return H, kpq, good
```

Listing 1: Full Ground Texture Algorithm Module (Ground.py)

Next Steps I

- Explore each algorithm and find out its hyperparameters and metrics.
- Find more research paper based on each algorithm.
- If the JetBot system is facing hardware constraints then find other methods to implement a ground localization algorithm.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Vielen Dank

für Ihre Aufmerksamkeit

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Quellen

References I

- Agrawal, Motilal, Kurt Konolige und M. R. Blas (2008). "CenSurE: Center surround extremas for realtime feature detection and matching". In: *ECCV*. ISBN: 9783540886921. DOI: 10.1007/978-3-540-88693-8_34.
- Alcantarilla, Pablo F., Jesús Nuevo und Adrien Bartoli (2013). "Fast explicit diffusion for accelerated features in nonlinear scale spaces". In: *BMVC*. DOI: 10.5244/C.27.13.
- Aqel, Mohammed O. A. u. a. (2016). "Optimal configuration of a downward-facing monocular camera for visual odometry". In: *Indian Journal of Science and Technology* 8.32. DOI: 10.17485/ijst/2015/v8i1/60546.
- Bay, Herbert, Tinne Tuytelaars und Luc Van Gool (2008). "SURF: Speeded Up Robust Features". In: *Computer Vision and Image Understanding* 110.3, S. 346–359. DOI: 10.1016/j.cviu.2007.09.014.

References II

- Bradski, Gary (2000). "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools*.
- Calonder, Michael u. a. (2010). "BRIEF: Binary Robust Independent Elementary Features". In: *ECCV*. ISBN: 9783642155604. DOI: 10.1007/978-3-642-15561-1_56.
- Chen, Xi, Anil S. Vempati und Paul Beardsley (2018). "StreetMap: Mapping and Localization on Ground Planes Using a Downward Facing Camera". In: *IROS*. DOI: 10.1109/IROS.2018.8593414.
- Cornick, Matthew u. a. (2016). "Localizing Ground Penetrating RADAR: A Step Toward Robust Autonomous Ground Vehicle Localization". In: *Journal of Field Robotics* 33.1, S. 82–102. DOI: 10.1002/rob.21532.
- Desouza, Guilherme N. und Avinash C. Kak (2002). "Vision for Mobile Robot Navigation: A Survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 24.2, S. 237–267. DOI: 10.1109/34.982903.

References III


Kelly, Alonzo u. a. (2007). "An Infrastructure-Free Automated Guided Vehicle Based on Computer Vision". In: *IEEE Robotics and Automation Magazine*. Bd. 14. 3, S. 24–35. DOI: 10.1109/MRA.2007.904800.


Kozak, Kevin und Matthew Alban (2016). "Ranger: A Ground-Facing Camera Based Localization System for Ground Vehicles". In: *IEEE/ION PLANS*. DOI: 10.1109/PLANS.2016.7479680.


Leutenegger, Stefan, Margarita Chli und Roland Y. Siegwart (2011). "BRISK: Binary Robust Invariant Scalable Keypoints". In: *ICCV*. DOI: 10.1109/ICCV.2011.6126542.


Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". In: *International Journal of Computer Vision* 60.2, S. 91–110. DOI: 10.1023/B:VISI.0000029664.99615.94.

References IV

- Matas, Jiri u. a. (2004). "Robust Wide Baseline Stereo from Maximally Stable Extremal Regions". In: *Image and Vision Computing*. Bd. 22. 10, S. 761–767. DOI: 10.1016/j.imavis.2004.02.006.
- Mur-Artal, Raul und Juan D. Tardós (2017). "ORB-SLAM2: An Open-Source SLAM System". In: *IEEE Transactions on Robotics*. Bd. 33. 5, S. 1255–1262. DOI: 10.1109/TRO.2017.2705103.
- Nakashima, Sho, Takashi Morio und Shao Mu (2019). "AKAZE-based Visual Odometry from Floor Images Supported by Acceleration Models". In: *IEEE Access*. Bd. 7, S. 31103–31109. DOI: 10.1109/ACCESS.2019.2902963.
- Pratt, William K. (2007). "Digital Image Processing". In: Wiley. ISBN: 9780471686590.
- Rublee, Ethan u. a. (2011). "ORB: An Efficient Alternative to SIFT or SURF". In: *ICCV*. DOI: 10.1109/ICCV.2011.6126544.

References V


Schmid, J. F., S. F. Simon und R. Mester (2019). "Features for Ground Texture Based Localization – A Survey". In: *Preprint*.


Zaman, M. (2007). "High Precision Relative Localization Using a Single Camera". In: *ICRA*. DOI: 10.1109/ROBOT.2007.363091.


Zhang, L., A. Finkelstein und S. Rusinkiewicz (2019). "High-Precision Localization Using Ground Texture". In: *ICRA*. DOI: 10.1109/ICRA.2019.8794185.